SQL — 150 Medium to Hard Coding Questions

A stronger SQL practice set covering joins, subqueries, CTEs, window functions, aggregation, dates, and analytical queries.

SQL dialect: MySQL 8.0. Sample schema used throughout the PDFs (MySQL 8.0):
employees(employee_id, first_name, last_name, department_id, salary, hire_date, manager_id)
departments(department_id, department_name)
customers(customer_id, customer_name, city, signup_date)
orders(order_id, customer_id, order_date, amount, status)
products(product_id, product_name, category, price)
order_items(order_id, product_id, quantity)

1. Select all employees

Question: Display every column from employees.

SQL:

```
SELECT * FROM employees;
```

Sample output: Returns all employee rows.

2. Select specific columns

Question: Display employee name and salary.

SQL:

```
SELECT first_name, last_name, salary FROM employees;
```

Sample output: Example: John Smith | 65000

3. Filter by salary

Question: Find employees earning more than 60000.

SQL:

```
SELECT first_name, salary FROM employees WHERE salary > 60000;
```

Sample output: Returns employees above 60000.

4. AND condition

Question: Find employees earning over 60000 in department 10.

SQL:

```
SELECT first_name, salary FROM employees WHERE salary > 60000 AND department_id = 10;
```

Sample output: Only rows satisfying both conditions.

5. OR condition

Question: Find employees in department 10 or 20.

SQL:

```
SELECT first_name, department_id FROM employees WHERE department_id IN (10,20);
```

Sample output: Departments 10 and 20.

6. BETWEEN

Question: Find salaries between 50000 and 80000.

SQL:

```
SELECT first_name, salary FROM employees WHERE salary BETWEEN 50000 AND 80000;
```

Sample output: Employees in that salary range.

7. IN

Question: Find customers from Hyderabad, Chennai, or Bengaluru.

SQL:

```
SELECT customer_name, city FROM customers WHERE city IN ('Hyderabad','Chennai','Bengaluru');
```

Sample output: Matching customers.

8. LIKE

Question: Find customers whose name starts with A.

SQL:

```
SELECT customer_name FROM customers WHERE customer_name LIKE 'A%';
```

Sample output: Names beginning with A.

9. IS NULL

Question: Find employees without a manager.

SQL:

```
SELECT first_name, manager_id FROM employees WHERE manager_id IS NULL;
```

Sample output: Employees with NULL manager_id.

10. ORDER BY

Question: List employees from highest to lowest salary.

SQL:

```
SELECT first_name, salary FROM employees ORDER BY salary DESC;
```

Sample output: Highest salary appears first.

11. DISTINCT

Question: List unique customer cities.

SQL:

```
SELECT DISTINCT city FROM customers ORDER BY city;
```

Sample output: One row per city.

12. COUNT

Question: Count all employees.

SQL:

```
SELECT COUNT(*) AS employee_count FROM employees;
```

Sample output: Example: 120

13. COUNT column

Question: Count employees having a manager.

SQL:

```
SELECT COUNT(manager_id) AS managed_employees FROM employees;
```

Sample output: Counts non-NULL manager_id values.

14. SUM

Question: Find total order amount.

SQL:

```
SELECT SUM(amount) AS total_sales FROM orders;
```

Sample output: Example: 1250000.00

15. AVG

Question: Find average salary.

SQL:

```
SELECT AVG(salary) AS avg_salary FROM employees;
```

Sample output: Example: 68250.50

16. MIN and MAX

Question: Find minimum and maximum product price.

SQL:

```
SELECT MIN(price) AS min_price, MAX(price) AS max_price FROM products;
```

Sample output: Example: 25.00 | 999.00

17. GROUP BY

Question: Count employees by department.

SQL:

```
SELECT department_id, COUNT(*) AS employee_count FROM employees GROUP BY department_id;
```

Sample output: One row per department.

18. GROUP BY SUM

Question: Find sales by order status.

SQL:

```
SELECT status, SUM(amount) AS total_amount FROM orders GROUP BY status;
```

Sample output: One row per status.

19. HAVING

Question: Find departments with more than 5 employees.

SQL:

```
SELECT department_id, COUNT(*) AS employee_count FROM employees GROUP BY department_id HAVING COUNT(*) > 5;
```

Sample output: Departments with >5 employees.

20. INNER JOIN

Question: Show employee names with department names.

SQL:

```
SELECT e.first_name, d.department_name FROM employees e JOIN departments d ON e.department_id=d.department_id;
```

Sample output: Employee and department pairs.

21. LEFT JOIN

Question: Show all departments and their employees if any.

SQL:

```
SELECT d.department_name, e.first_name FROM departments d LEFT JOIN employees e ON d.department_id=e.department_id;
```

Sample output: Departments without employees are retained.

22. RIGHT JOIN

Question: Show all employees and matching departments using RIGHT JOIN.

SQL:

```
SELECT e.first_name, d.department_name FROM departments d RIGHT JOIN employees e ON d.department_id=e.department_id;
```

Sample output: All employees are retained.

23. JOIN orders customers

Question: Show order id, customer name and amount.

SQL:

```
SELECT o.order_id,c.customer_name,o.amount FROM orders o JOIN customers c ON o.customer_id=c.customer_id;
```

Sample output: Order details with customer.

24. Date filter

Question: Find orders placed in 2026.

SQL:

```
SELECT * FROM orders WHERE order_date >= '2026-01-01' AND order_date < '2027-01-01';
```

Sample output: Orders from 2026.

25. DATE

Question: Show order date without time.

SQL:

```
SELECT order_id, DATE(order_date) AS order_day FROM orders;
```

Sample output: Date-only values.

26. YEAR

Question: Count orders by year.

SQL:

```
SELECT YEAR(order_date) AS order_year, COUNT(*) AS orders FROM orders GROUP BY YEAR(order_date);
```

Sample output: One row per year.

27. MONTH

Question: Find monthly sales.

SQL:

```
SELECT YEAR(order_date) AS yr, MONTH(order_date) AS mon, SUM(amount) AS sales FROM orders GROUP BY YEAR(order_date),
 MONTH(order_date);
```

Sample output: Monthly totals.

28. CASE

Question: Classify salaries as High/Medium/Low.

SQL:

```
SELECT first_name,salary,CASE WHEN salary>=80000 THEN 'High' WHEN salary>=50000 THEN 'Medium' ELSE 'Low' END AS sala
ry_band FROM employees;
```

Sample output: Salary band per employee.

29. COALESCE

Question: Show manager id or 0 if missing.

SQL:

```
SELECT first_name, COALESCE(manager_id,0) AS manager_id FROM employees;
```

Sample output: NULL manager IDs become 0.

30. ROUND

Question: Round order amounts to whole numbers.

SQL:

```
SELECT order_id, ROUND(amount,0) AS rounded_amount FROM orders;
```

Sample output: Rounded amount.

31. LIMIT

Question: Find top 5 highest-paid employees.

SQL:

```
SELECT first_name,salary FROM employees ORDER BY salary DESC LIMIT 5;
```

Sample output: Five highest salaries.

32. OFFSET

Question: Skip first 5 employees ordered by salary.

SQL:

```
SELECT first_name,salary FROM employees ORDER BY salary DESC LIMIT 5 OFFSET 5;
```

Sample output: Rows 6-10 by salary.

33. CONCAT

Question: Create full employee names.

SQL:

```
SELECT CONCAT(first_name,' ',last_name) AS full_name FROM employees;
```

Sample output: Example: John Smith

34. LOWER

Question: Convert customer names to lowercase.

SQL:

```
SELECT LOWER(customer_name) AS customer_name FROM customers;
```

Sample output: Lowercase names.

35. UPPER

Question: Convert department names to uppercase.

SQL:

```
SELECT UPPER(department_name) FROM departments;
```

Sample output: Uppercase names.

36. LENGTH

Question: Find customers whose names have more than 10 characters.

SQL:

```
SELECT customer_name FROM customers WHERE LENGTH(customer_name)>10;
```

Sample output: Long customer names.

37. TRIM

Question: Remove surrounding spaces from names.

SQL:

```
SELECT TRIM(customer_name) FROM customers;
```

Sample output: Trimmed names.

38. REPLACE

Question: Replace Hyderabad with Hyd.

SQL:

```
SELECT customer_name, REPLACE(city,'Hyderabad','Hyd') AS city FROM customers;
```

Sample output: Modified city label.

39. NULLIF

Question: Avoid division by zero.

SQL:

```
SELECT amount/NULLIF(quantity,0) AS unit_amount FROM order_items;
```

Sample output: NULL when quantity is zero.

40. UNION

Question: Combine customer cities and employee department IDs as text.

SQL:

```
SELECT city AS value FROM customers UNION SELECT CAST(department_id AS CHAR) FROM employees;
```

Sample output: Distinct combined values.

41. UNION ALL

Question: Combine two result sets preserving duplicates.

SQL:

```
SELECT city FROM customers UNION ALL SELECT city FROM customers;
```

Sample output: Every row appears twice.

42. Subquery

Question: Find employees earning above average salary.

SQL:

```
SELECT first_name,salary FROM employees WHERE salary>(SELECT AVG(salary) FROM employees);
```

Sample output: Above-average employees.

43. MAX subquery

Question: Find employees with maximum salary.

SQL:

```
SELECT first_name,salary FROM employees WHERE salary=(SELECT MAX(salary) FROM employees);
```

Sample output: Highest-paid employees.

44. EXISTS

Question: Find customers who have at least one order.

SQL:

```
SELECT c.customer_id,c.customer_name FROM customers c WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id=c.cus
tomer_id);
```

Sample output: Customers with orders.

45. NOT EXISTS

Question: Find customers with no orders.

SQL:

```
SELECT c.customer_id,c.customer_name FROM customers c WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id=c
.customer_id);
```

Sample output: Customers without orders.

46. IN subquery

Question: Find customers who placed completed orders.

SQL:

```
SELECT customer_name FROM customers WHERE customer_id IN (SELECT customer_id FROM orders WHERE status='Completed');
```

Sample output: Customers with completed orders.

47. UPDATE

Question: Increase salaries by 10% for department 10.

SQL:

```
UPDATE employees SET salary=salary*1.10 WHERE department_id=10;
```

Sample output: Department 10 salaries increase 10%.

48. DELETE

Question: Delete cancelled orders.

SQL:

```
DELETE FROM orders WHERE status='Cancelled';
```

Sample output: Cancelled rows removed.

49. INSERT

Question: Insert a new department.

SQL:

```
INSERT INTO departments(department_id,department_name) VALUES (50,'Analytics');
```

Sample output: One department inserted.

50. CREATE TABLE

Question: Create a simple skills table.

SQL:

```
CREATE TABLE skills(skill_id INT PRIMARY KEY, skill_name VARCHAR(100) NOT NULL);
```

Sample output: Table created.

51. PRIMARY KEY

Question: Create employees table with a primary key.

SQL:

```
CREATE TABLE demo_employee(employee_id INT PRIMARY KEY, first_name VARCHAR(50));
```

Sample output: employee_id uniquely identifies rows.

52. FOREIGN KEY

Question: Create order table with customer foreign key.

SQL:

```
CREATE TABLE demo_order(order_id INT PRIMARY KEY, customer_id INT, FOREIGN KEY(customer_id) REFERENCES customers(cus
tomer_id));
```

Sample output: Referential integrity enforced.

53. UNIQUE

Question: Create users with unique email.

SQL:

```
CREATE TABLE users(user_id INT PRIMARY KEY,email VARCHAR(150) UNIQUE);
```

Sample output: Duplicate emails prevented.

54. NOT NULL

Question: Require product name.

SQL:

```
CREATE TABLE demo_product(product_id INT PRIMARY KEY,product_name VARCHAR(100) NOT NULL);
```

Sample output: Product name cannot be NULL.

55. DEFAULT

Question: Give status a default value.

SQL:

```
CREATE TABLE demo_order2(order_id INT PRIMARY KEY,status VARCHAR(20) DEFAULT 'Pending');
```

Sample output: New rows default to Pending.

56. ALTER TABLE

Question: Add phone column to customers.

SQL:

```
ALTER TABLE customers ADD COLUMN phone VARCHAR(20);
```

Sample output: phone column added.

57. DROP COLUMN

Question: Remove phone column.

SQL:

```
ALTER TABLE customers DROP COLUMN phone;
```

Sample output: phone column removed.

58. View

Question: Create a view for high earners.

SQL:

```
CREATE VIEW high_earners AS SELECT * FROM employees WHERE salary>80000;
```

Sample output: View created.

59. CTE

Question: Use a CTE for average salary.

SQL:

```
WITH avg_sal AS (SELECT AVG(salary) avg_salary FROM employees) SELECT e.first_name,e.salary FROM employees e CROSS J
OIN avg_sal WHERE e.salary>avg_sal.avg_salary;
```

Sample output: Above-average employees.

60. ROW_NUMBER

Question: Number employees by salary.

SQL:

```
SELECT first_name,salary,ROW_NUMBER() OVER(ORDER BY salary DESC) AS rn FROM employees;
```

Sample output: Sequential salary ranking.

61. RANK

Question: Rank employees by salary.

SQL:

```
SELECT first_name,salary,RANK() OVER(ORDER BY salary DESC) AS salary_rank FROM employees;
```

Sample output: Ties receive same rank.

62. DENSE_RANK

Question: Dense-rank salaries.

SQL:

```
SELECT first_name,salary,DENSE_RANK() OVER(ORDER BY salary DESC) AS salary_rank FROM employees;
```

Sample output: No gaps after ties.

63. PARTITION BY

Question: Rank employees within department.

SQL:

```
SELECT first_name,department_id,salary,DENSE_RANK() OVER(PARTITION BY department_id ORDER BY salary DESC) AS rnk FRO
M employees;
```

Sample output: Rank resets per department.

64. LAG

Question: Compare salary with previous employee.

SQL:

```
SELECT first_name,salary,LAG(salary) OVER(ORDER BY salary) AS previous_salary FROM employees;
```

Sample output: Previous salary shown.

65. LEAD

Question: Compare salary with next employee.

SQL:

```
SELECT first_name,salary,LEAD(salary) OVER(ORDER BY salary) AS next_salary FROM employees;
```

Sample output: Next salary shown.

66. Running total

Question: Calculate cumulative order sales.

SQL:

```
SELECT order_date,amount,SUM(amount) OVER(ORDER BY order_date,order_id) AS running_sales FROM orders;
```

Sample output: Cumulative sales.

67. Moving average

Question: Calculate 3-order moving average.

SQL:

```
SELECT order_id,amount,AVG(amount) OVER(ORDER BY order_id ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS moving_avg FR
OM orders;
```

Sample output: Three-row moving average.

68. Duplicate emails

Question: Find duplicate emails in a users table.

SQL:

```
SELECT email,COUNT(*) FROM users GROUP BY email HAVING COUNT(*)>1;
```

Sample output: Duplicate email values.

69. Duplicate customer names

Question: Find duplicate customer names.

SQL:

```
SELECT customer_name,COUNT(*) AS cnt FROM customers GROUP BY customer_name HAVING COUNT(*)>1;
```

Sample output: Repeated names.

70. Second highest salary

Question: Find second-highest distinct salary.

SQL:

```
SELECT MAX(salary) AS second_highest FROM employees WHERE salary<(SELECT MAX(salary) FROM employees);
```

Sample output: Second-highest salary.

71. Nth highest salary

Question: Find the 3rd highest distinct salary.

SQL:

```
SELECT salary FROM (SELECT DISTINCT salary,DENSE_RANK() OVER(ORDER BY salary DESC) rnk FROM employees) x WHERE rnk=3
;
```

Sample output: Third-highest distinct salary.

72. Employees above department average

Question: Find employees earning above their department average.

SQL:

```
SELECT e.first_name,e.department_id,e.salary FROM employees e WHERE e.salary>(SELECT AVG(x.salary) FROM employees x
WHERE x.department_id=e.department_id);
```

Sample output: Above departmental average.

73. Department max salary

Question: Find highest salary per department.

SQL:

```
SELECT department_id,MAX(salary) AS max_salary FROM employees GROUP BY department_id;
```

Sample output: One maximum per department.

74. Departments with no employees

Question: Find departments without employees.

SQL:

```
SELECT d.department_name FROM departments d LEFT JOIN employees e ON d.department_id=e.department_id WHERE e.employe
e_id IS NULL;
```

Sample output: Unused departments.

75. Customers with total spend

Question: Calculate spend per customer.

SQL:

```
SELECT customer_id,SUM(amount) AS total_spend FROM orders GROUP BY customer_id;
```

Sample output: Total spend per customer.

76. Top customer

Question: Find customer with highest total spend.

SQL:

```
SELECT customer_id,SUM(amount) total_spend FROM orders GROUP BY customer_id ORDER BY total_spend DESC LIMIT 1;
```

Sample output: Highest-spending customer.

77. Products never ordered

Question: Find products absent from order_items.

SQL:

```
SELECT p.product_id,p.product_name FROM products p LEFT JOIN order_items oi ON p.product_id=oi.product_id WHERE oi.p
roduct_id IS NULL;
```

Sample output: Never-ordered products.

78. Order item revenue

Question: Calculate revenue by product.

SQL:

```
SELECT oi.product_id,SUM(oi.quantity*p.price) revenue FROM order_items oi JOIN products p ON oi.product_id=p.product
_id GROUP BY oi.product_id;
```

Sample output: Revenue per product.

79. Category revenue

Question: Calculate revenue by product category.

SQL:

```
SELECT p.category,SUM(oi.quantity*p.price) revenue FROM order_items oi JOIN products p ON oi.product_id=p.product_id
 GROUP BY p.category;
```

Sample output: Revenue per category.

80. Completed sales

Question: Sum only completed orders.

SQL:

```
SELECT SUM(amount) FROM orders WHERE status='Completed';
```

Sample output: Completed sales total.

81. Conditional aggregation

Question: Count completed and cancelled orders in one row.

SQL:

```
SELECT SUM(status='Completed') completed,SUM(status='Cancelled') cancelled FROM orders;
```

Sample output: Two status counts.

82. CASE aggregation

Question: Count salary bands.

SQL:

```
SELECT SUM(salary>=80000) high,SUM(salary BETWEEN 50000 AND 79999) medium,SUM(salary<50000) low FROM employees;
```

Sample output: Counts by salary band.

83. Date difference

Question: Find employee tenure in days.

SQL:

```
SELECT first_name,DATEDIFF(CURDATE(),hire_date) AS tenure_days FROM employees;
```

Sample output: Tenure in days.

84. Month name

Question: Show order month names.

SQL:

```
SELECT MONTHNAME(order_date) AS month_name,COUNT(*) FROM orders GROUP BY MONTH(order_date),MONTHNAME(order_date);
```

Sample output: Order counts by month.

85. Current year

Question: Find employees hired this year.

SQL:

```
SELECT * FROM employees WHERE YEAR(hire_date)=YEAR(CURDATE());
```

Sample output: Employees hired this year.

86. Weekend orders

Question: Find orders placed on Saturday/Sunday.

SQL:

```
SELECT order_id,order_date FROM orders WHERE DAYOFWEEK(order_date) IN (1,7);
```

Sample output: Weekend orders.

87. Case-insensitive search

Question: Find names containing 'an'.

SQL:

```
SELECT customer_name FROM customers WHERE LOWER(customer_name) LIKE '%an%';
```

Sample output: Matching names.

88. Join three tables

Question: Show customer, order and product information.

SQL:

```
SELECT c.customer_name,o.order_id,p.product_name,oi.quantity FROM customers c JOIN orders o ON c.customer_id=o.custo
mer_id JOIN order_items oi ON o.order_id=oi.order_id JOIN products p ON oi.product_id=p.product_id;
```

Sample output: Combined transaction details.

89. Order count per customer

Question: Count orders for every customer, including zero.

SQL:

```
SELECT c.customer_name,COUNT(o.order_id) order_count FROM customers c LEFT JOIN orders o ON c.customer_id=o.customer
_id GROUP BY c.customer_id,c.customer_name;
```

Sample output: Every customer with order count.

90. Average order by city

Question: Find average order amount by customer city.

SQL:

```
SELECT c.city,AVG(o.amount) avg_order FROM customers c JOIN orders o ON c.customer_id=o.customer_id GROUP BY c.city;
```

Sample output: Average amount per city.

91. Employees per manager

Question: Count direct reports.

SQL:

```
SELECT manager_id,COUNT(*) AS reports FROM employees WHERE manager_id IS NOT NULL GROUP BY manager_id;
```

Sample output: Direct reports per manager.

92. Manager names

Question: Show employee and manager names.

SQL:

```
SELECT e.first_name employee,m.first_name manager FROM employees e LEFT JOIN employees m ON e.manager_id=m.employee_
id;
```

Sample output: Employee-manager pairs.

93. Self join salary comparison

Question: Find employees earning more than their manager.

SQL:

```
SELECT e.first_name employee,e.salary employee_salary,m.first_name manager,m.salary manager_salary FROM employees e
JOIN employees m ON e.manager_id=m.employee_id WHERE e.salary>m.salary;
```

Sample output: Employees earning more than manager.

94. COALESCE join

Question: Show department name or 'Unassigned'.

SQL:

```
SELECT e.first_name,COALESCE(d.department_name,'Unassigned') FROM employees e LEFT JOIN departments d ON e.departmen
t_id=d.department_id;
```

Sample output: Department label for every employee.

95. SQL comment practice

Question: Write a query with an explanatory comment.

SQL:

```
-- Find active orders
SELECT * FROM orders WHERE status='Completed';
```

Sample output: Completed orders.

96. Top 3 per department

Question: Return the three highest-paid employees in each department.

SQL:

```
SELECT * FROM (SELECT e.*,ROW_NUMBER() OVER(PARTITION BY department_id ORDER BY salary DESC) rn FROM employees e) x
WHERE rn<=3;
```

Sample output: Up to three employees per department.

97. Latest order per customer

Question: Return each customer's latest order.

SQL:

```
SELECT * FROM (SELECT o.*,ROW_NUMBER() OVER(PARTITION BY customer_id ORDER BY order_date DESC,order_id DESC) rn FROM
 orders o) x WHERE rn=1;
```

Sample output: One latest order per customer.

98. First order per customer

Question: Return each customer's first order.

SQL:

```
SELECT * FROM (SELECT o.*,ROW_NUMBER() OVER(PARTITION BY customer_id ORDER BY order_date,order_id) rn FROM orders o)
 x WHERE rn=1;
```

Sample output: One first order per customer.

99. Customers ordering in consecutive months

Question: Find customers with orders in at least two distinct months.

SQL:

```
SELECT customer_id FROM orders GROUP BY customer_id HAVING COUNT(DISTINCT DATE_FORMAT(order_date,'%Y-%m'))>=2;
```

Sample output: Customers active in multiple months.

100. Monthly growth

Question: Calculate month-over-month sales growth.

SQL:

```
WITH m AS (SELECT DATE_FORMAT(order_date,'%Y-%m') mon,SUM(amount) sales FROM orders GROUP BY DATE_FORMAT(order_date,
'%Y-%m')) SELECT mon,sales,LAG(sales) OVER(ORDER BY mon) prev_sales,(sales-LAG(sales) OVER(ORDER BY mon))/NULLIF(LAG
(sales) OVER(ORDER BY mon),0)*100 growth_pct FROM m;
```

Sample output: Monthly sales with growth percentage.

101. Pareto customers

Question: Find customers contributing to cumulative 80% of sales.

SQL:

```
WITH x AS (SELECT customer_id,SUM(amount) sales FROM orders GROUP BY customer_id), y AS (SELECT *,SUM(sales) OVER(OR
DER BY sales DESC)/SUM(sales) OVER()*100 cum_pct FROM x) SELECT * FROM y WHERE cum_pct<=80;
```

Sample output: Customers within cumulative 80%.

102. Median salary

Question: Calculate median salary using window functions.

SQL:

```
WITH x AS (SELECT salary,ROW_NUMBER() OVER(ORDER BY salary) rn,COUNT(*) OVER() n FROM employees) SELECT AVG(salary)
median_salary FROM x WHERE rn IN (FLOOR((n+1)/2),FLOOR((n+2)/2));
```

Sample output: Median salary.

103. Gaps in order dates

Question: Find dates with no orders between min and max date.

SQL:

```
WITH RECURSIVE d AS (SELECT MIN(DATE(order_date)) dt FROM orders UNION ALL SELECT dt+INTERVAL 1 DAY FROM d WHERE dt<
(SELECT MAX(DATE(order_date)) FROM orders)) SELECT d.dt FROM d LEFT JOIN (SELECT DISTINCT DATE(order_date) dt FROM o
rders)o ON d.dt=o.dt WHERE o.dt IS NULL;
```

Sample output: Missing order dates.

104. Recursive hierarchy

Question: Show employee hierarchy from top-level managers.

SQL:

```
WITH RECURSIVE org AS (SELECT employee_id,first_name,manager_id,0 lvl FROM employees WHERE manager_id IS NULL UNION
ALL SELECT e.employee_id,e.first_name,e.manager_id,o.lvl+1 FROM employees e JOIN org o ON e.manager_id=o.employee_id
) SELECT * FROM org ORDER BY lvl,employee_id;
```

Sample output: Hierarchy levels.

105. Running customer spend

Question: Calculate cumulative spend per customer.

SQL:

```
SELECT customer_id,order_date,amount,SUM(amount) OVER(PARTITION BY customer_id ORDER BY order_date,order_id) cumulat
ive_spend FROM orders;
```

Sample output: Running spend.

106. Rank products by revenue

Question: Rank products by calculated revenue.

SQL:

```
WITH r AS (SELECT p.product_id,p.product_name,SUM(oi.quantity*p.price) revenue FROM products p JOIN order_items oi O
N p.product_id=oi.product_id GROUP BY p.product_id,p.product_name) SELECT *,DENSE_RANK() OVER(ORDER BY revenue DESC)
 rnk FROM r;
```

Sample output: Revenue ranking.

107. Category top product

Question: Find the highest-revenue product in each category.

SQL:

```
WITH r AS (SELECT p.category,p.product_id,p.product_name,SUM(oi.quantity*p.price) revenue FROM products p JOIN order
_items oi ON p.product_id=oi.product_id GROUP BY p.category,p.product_id,p.product_name),x AS (SELECT *,ROW_NUMBER()
 OVER(PARTITION BY category ORDER BY revenue DESC) rn FROM r) SELECT * FROM x WHERE rn=1;
```

Sample output: Top product per category.

108. Customers with no orders last 90 days

Question: Find customers inactive in the last 90 days.

SQL:

```
SELECT c.* FROM customers c LEFT JOIN orders o ON c.customer_id=o.customer_id AND o.order_date>=CURRENT_DATE-INTERVA
L 90 DAY WHERE o.order_id IS NULL;
```

Sample output: Inactive customers.

109. Repeat customers

Question: Find customers with at least two completed orders.

SQL:

```
SELECT customer_id FROM orders WHERE status='Completed' GROUP BY customer_id HAVING COUNT(*)>=2;
```

Sample output: Repeat customers.

110. Order conversion

Question: Calculate percentage of orders that are completed.

SQL:

```
SELECT 100*SUM(status='Completed')/COUNT(*) AS completion_pct FROM orders;
```

Sample output: Completion percentage.

111. Salary quartiles

Question: Assign employees to four salary quartiles.

SQL:

```
SELECT first_name,salary,NTILE(4) OVER(ORDER BY salary) quartile FROM employees;
```

Sample output: Quartile 1-4.

112. Department salary share

Question: Calculate each department's share of total payroll.

SQL:

```
WITH d AS (SELECT department_id,SUM(salary) payroll FROM employees GROUP BY department_id) SELECT department_id,payr
oll,100*payroll/SUM(payroll) OVER() pct_total FROM d;
```

Sample output: Payroll share.

113. Customer lifetime value

Question: Calculate completed-order lifetime value per customer.

SQL:

```
SELECT customer_id,SUM(CASE WHEN status='Completed' THEN amount ELSE 0 END) ltv FROM orders GROUP BY customer_id;
```

Sample output: Completed sales per customer.

114. Product basket count

Question: Count distinct orders containing each product.

SQL:

```
SELECT product_id,COUNT(DISTINCT order_id) orders_count FROM order_items GROUP BY product_id;
```

Sample output: Distinct order count per product.

115. Average items per order

Question: Calculate average number of items per order.

SQL:

```
SELECT AVG(item_count) FROM (SELECT order_id,SUM(quantity) item_count FROM order_items GROUP BY order_id)x;
```

Sample output: Average item quantity per order.